

Project 1: Temperature Monitor with Alert

STM32F103C8T6 Blue Pill + 37-in-1 Sensor Kit

What You're Building

A system that reads temperature from the DHT11 sensor every 2 seconds, sends the reading to your laptop over UART (you see it in a terminal window), and when the temperature crosses a threshold (say 30°C), it turns on an LED and triggers the buzzer from your kit.

Simple concept. But building it teaches you 80% of what embedded engineering is about.

Your Setup

- **Editor:** VS Code
- **Toolchain:** STM32 CLI tools (stm32cubeclt or similar) for compiling, flashing, and debugging
- **Connection:** ST-Link V2 connected via USB to your PC, wired to the Blue Pill's SWD pins
- **Breadboard** for clean, stable connections between the Blue Pill and your sensor kit components

How the physical setup works:

Plug your Blue Pill into the breadboard — its header pins slot into the breadboard holes. The breadboard's internal rows are connected horizontally, so anything you plug into the same row as a Blue Pill pin is electrically connected to it. Use the two long rails running along the edges for power (3.3V) and ground (GND) — run a jumper wire from the Blue Pill's 3.3V pin to the positive rail and GND to the negative rail. Then all your components can tap power from these rails instead of fighting over the Blue Pill's limited power/ground pins. Sensor modules from your kit plug directly into the breadboard or connect via jumper wires.

The ST-Link V2 is already connected to your Blue Pill via 4 wires:

- SWDIO → SWDIO
- SWCLK → SWCLK
- GND → GND
- 3.3V → 3.3V

The ST-Link's USB side plugs into your PC. This single connection handles both power (3.3V to the Blue Pill) and programming/debugging (SWD interface). Your Blue Pill is powered through the ST-Link — no separate power source needed.

Hardware Connections

Using female-to-female jumper wires, connect directly to the Blue Pill header pins:

DHT11 Sensor → Blue Pill (via breadboard):

VCC → 3.3V rail on breadboard

GND → GND rail on breadboard

DATA → PA1

LED → Blue Pill (via breadboard):

Plug the LED into the breadboard. Connect a 220Ω resistor from PA2's row to the LED's anode (long leg) row. Connect the LED's cathode (short leg) row to the GND rail.

Buzzer → Blue Pill (via breadboard):

VCC → 3.3V rail on breadboard

GND → GND rail on breadboard

SIGNAL → PA3

Important: Use the breadboard's power rails. Run one jumper from the Blue Pill's 3.3V pin to the breadboard's positive (+) rail, and one from GND to the negative (–) rail. Then connect all component VCC wires to the positive rail and all GND wires to the negative rail. This keeps things organized and you won't run out of power/ground pins on the Blue Pill.

Software Setup

Step 1: Set up VS Code for STM32 development

You need a few things working in VS Code:

The ARM GCC cross-compiler — this compiles your C code into machine code that runs on the ARM Cortex-M3 chip, not on your PC. The STM32 CLI tools should have installed this. Verify by opening your terminal in VS Code and running `arm-none-eabi-gcc --version`. If it prints a version number, you're good.

The STM32 CLI tools — you're already using these. The key commands you'll need are for building (compiling your project), flashing (sending the compiled binary to the Blue Pill through the ST-Link), and debugging (stepping through code running on the chip from VS Code). Familiarize yourself with the CLI commands for each of these operations. Run the help command to see all available options.

VS Code extensions — install the Cortex-Debug extension for debugging, and the C/C++ extension from Microsoft for syntax highlighting and IntelliSense. These let you set breakpoints in your C code and step through it while it's running on the actual chip — exactly like debugging Java in IntelliJ, except the code is running on a microcontroller connected via your ST-Link.

STM32CubeMX — this is a separate free tool from ST (not an IDE, just a configurator). You use it to graphically configure which pins do what, enable peripherals like UART and timers, set clock speeds, and then it generates the initialization C code for you. Download it from st.com. You'll use CubeMX to generate the project, then open and edit the generated code in VS Code.

Step 2: Create the project with STM32CubeMX

Open CubeMX, start a new project, and select STM32F103C8T6 as your chip. You'll see a diagram of the chip with all its pins. Configure:

- **PA1** as GPIO_Input — this is your DHT11 data line
- **PA2** as GPIO_Output — this is your LED control
- **PA3** as GPIO_Output — this is your buzzer control
- **USART1** — enable it on PA9 (TX) and PA10 (RX) at 115200 baud rate. This is how you'll send text data from the chip to your PC
- **TIM2** — enable this timer. You need it because the DHT11 protocol requires timing precision at the microsecond level, and the default HAL_Delay function only works in milliseconds. You'll configure TIM2's prescaler so that each counter tick equals 1 microsecond. To do this: set the prescaler to $(\text{SystemCoreClock} / 1000000) - 1$ and the counter period to 65535

In the Project Manager tab: give your project a name, set the toolchain to "Makefile" (since you're using CLI tools and VS Code, not an IDE), and choose your project folder location. Click "Generate Code."

CubeMX creates a complete project folder with a Makefile, all the HAL driver source files, and a main.c with initialization code already written.

Step 3: Open the project in VS Code

Open the generated project folder in VS Code. Look at the file structure:

- **Core/Src/main.c** — your main file, where you'll write your application code

- **Core/Inc/** — header files
- **Drivers/** — the STM32 HAL library source code (read these to understand the API)
- **Makefile** — build script that compiles everything

Open `main.c`. You'll see CubeMX generated initialization functions that configure all the hardware you set up. At the bottom is an empty infinite while loop — this is where your application code goes. Everything between the `USER CODE BEGIN` and `USER CODE END` comments is yours to edit. Don't write outside these markers — if you regenerate from CubeMX later, it will overwrite anything outside them.

Step 4: Build and verify the toolchain works

Before writing any application code, make sure the basic toolchain works. Open the VS Code terminal, navigate to your project folder, and run `make`. It should compile without errors and produce a `.bin` or `.elf` file in the build output folder.

Then flash the empty program to your Blue Pill using the STM32 CLI flash command, pointing it at the compiled binary and specifying the ST-Link as the programmer. If it flashes successfully, your entire toolchain (compiler → binary → ST-Link → chip) is working. This is your "hello world" moment — the chip is now running your code, even though that code does nothing yet.

Step 5: Write the DHT11 reading code

Create two new files: `Core/Src/dht11.c` and `Core/Inc/dht11.h`. Add `dht11.c` to your Makefile's source file list so it gets compiled.

What your header file needs: A struct definition that holds temperature and humidity as floats, and a function declaration for reading the sensor. Think of this as your Java interface — it defines what the outside world can call.

What your driver source file needs to do:

Microsecond delay function: The HAL's built-in delay only works in milliseconds, but the DHT11 protocol needs microsecond precision. Write a function that uses TIM2 to create precise microsecond delays. The approach: reset the timer counter to zero, then spin in a tight loop checking the counter value until it reaches your target number of microseconds. Look at the HAL timer macros — there are macros specifically for setting and getting the timer counter value. Find them by reading the timer HAL header file in your Drivers folder.

Pin mode switching functions: The DHT11 data pin needs to be an output when you send the start signal, then switch to an input when you read the sensor's response. Write two helper functions that reconfigure PA1's direction at runtime. Look at how CubeMX generated the GPIO initialization code in `main.c` — you'll use the same `GPIO_InitTypeDef` struct and `HAL_GPIO_Init` function, just with different Mode values.

The main read function: This implements the DHT11 protocol. Read the DHT11 datasheet (Google "DHT11 datasheet PDF") — page 5 has the timing diagram that shows exactly what signals happen on the wire. Your function needs to:

1. Switch the data pin to output mode and pull it LOW for 18 milliseconds — this is the "start signal" telling the sensor to wake up
2. Release the pin (set HIGH) and wait about 20 microseconds
3. Switch the data pin to input mode and wait for the sensor's response
4. The sensor pulls LOW for about 80 microseconds then HIGH for about 80 microseconds — this is its acknowledgment
5. Then the sensor sends 40 bits of data, one at a time. Each bit starts with a LOW period (about 50 microseconds). Then a HIGH period follows — if the HIGH period is short (about 26 microseconds) the bit is a 0, if long (about 70 microseconds) the bit is a 1
6. You need to measure the HIGH duration for each bit and assemble 40 bits into 5 bytes
7. The 5 bytes are: humidity integer, humidity decimal, temperature integer, temperature decimal, and a checksum. Verify the checksum by adding the first four bytes — the result should equal the fifth byte

The bitwise assembly hint: You're filling 5 bytes, 8 bits each, from a stream of 40 individual bits. Think about which byte you're currently filling (bit number divided by 8) and which position within that byte (bit number modulo 8, but remember — the first bit received is the most significant bit, bit 7). The OR-equals operator combined with a left shift is the pattern you need. You've already seen this exact pattern in the FreeRTOS source code when it manipulates status flags.

Step 6: Write the main loop

What your main loop needs to do:

Call your DHT11 read function to get the current temperature and humidity. Format the readings into a human-readable string — look up the `sprintf` function from `stdio.h`, it works exactly like `String.format` in Java. Then transmit that string over UART to your PC — find the HAL UART transmit function in the UART HAL header file and figure out its parameters (it takes the UART handle, a pointer to your data buffer, the length, and a timeout).

After transmitting, check the temperature value against your threshold. If it exceeds 30°C, use the HAL GPIO write function to set PA2 HIGH (LED on) and PA3 HIGH (buzzer on). If below threshold, set both LOW. Then call `HAL_Delay(2000)` to wait 2 seconds before the next reading.

Seeing the UART output on your PC: You need a serial terminal program. You can use the VS Code serial monitor extension, or PuTTY, or any terminal that can open a serial port. But here's the catch — since your Blue Pill connects to your PC only through the ST-Link, and the ST-Link's USB interface is for SWD debugging not UART, you have two options:

Option A: Use SWO trace / semihosting through the ST-Link. This lets you use printf-style output that goes through the SWD debug connection instead of UART. Look up "STM32 SWO trace" or "STM32 semihosting printf" — it requires some configuration in your debug setup but no extra hardware.

Option B: Buy a cheap USB-to-UART adapter (CP2102 or CH340 based, about Rs 200-300 on Daraz). Connect its RX pin to PA9 (your Blue Pill's TX), its TX to PA10, and GND to GND. Plug its USB into your PC. It shows up as a serial port that you open in your terminal program at 115200 baud. This is the cleaner approach and useful for many future projects.

Option C: If your Blue Pill's USB port works, you can configure USB CDC (virtual COM port) in CubeMX and send data through the Blue Pill's own USB connection. But this is more complex to set up — try Option A or B first.

Step 7: Build, flash, and run

In your VS Code terminal, run `make` to compile. Fix any errors — the compiler error messages will point you to exact line numbers. Once it compiles clean, use the STM32 CLI flash command to send the binary to the chip. The program starts running immediately after flashing.

Open your serial terminal (whichever option you chose above) and watch the temperature readings appear every 2 seconds. Hold the DHT11 sensor in your hand — your body heat should push the reading above 30°C. The LED lights up and the buzzer sounds.

If it doesn't work the first time, that's normal. This is embedded development. Use the debugger — set breakpoints in your code from VS Code using the Cortex-Debug extension, step through line by line, and inspect variable values to find where things go wrong. This is the same debugging process you use in Java, just with hardware in the loop.

What This Project Teaches You

Concept	Where you learned it
GPIO Output	Controlling LED and buzzer
GPIO Input	Reading DHT11 data pin
Digital protocol	Parsing DHT11's 40-bit custom protocol
Timers	Microsecond delays for protocol timing
UART	Sending data to your laptop
Structs in C	Your DHT11 data struct

Concept	Where you learned it
Bitwise operations	Assembling bits into bytes
HAL library patterns	GPIO, UART, Timer HAL functions
Peripheral configuration	CubeMX pin setup
Cross-compilation	Building ARM code on your PC
Flash/debug workflow	ST-Link CLI → chip
Conditional hardware control	Threshold-based LED/buzzer activation

When You Get Stuck

1. **make fails with "arm-none-eabi-gcc not found"** → Your ARM toolchain isn't in PATH. Find where the STM32 CLI tools installed it and add to your system PATH, or point to it in the Makefile.
2. **Flash fails with "No ST-Link detected"** → Check USB connection. Try unplugging and replugging the ST-Link. Make sure drivers are installed (ST provides USB drivers for Windows). On Linux you may need udev rules for ST-Link permissions.
3. **Flash succeeds but nothing happens** → Your code might be running but you can't see output. Set a breakpoint in main.c and debug — if the debugger hits the breakpoint, the chip is running your code. The problem is likely in your UART output setup, not the code logic.
4. **DHT11 reads all zeros** → Most common issue. Check DATA pin wiring is on PA1. Try adding a 10KΩ pull-up resistor between the DATA line and 3.3V (some DHT11 modules have this built in, some don't). Also verify your microsecond delay function is actually producing accurate microsecond delays — if the timer prescaler is wrong, all your protocol timing will be off.
5. **DHT11 reads once then stops** → You might not be waiting long enough between reads. DHT11 needs at minimum 1 second between readings. Your 2-second delay should be fine, but verify HAL_Delay is actually working by toggling the LED in the loop as a visual heartbeat.
6. **Buzzer doesn't sound** → Check if yours is an active buzzer (has a built-in oscillator, sounds with just HIGH voltage) or a passive buzzer (needs a PWM signal to produce sound). If passive, you'll need to use a timer in PWM mode instead of simple GPIO output. Most 37-in-1 kits include an active buzzer — check the module label.

7. **LED doesn't light** → Check polarity (long leg is anode, goes toward PA2 through the resistor). Check that the resistor is actually 220Ω (red-red-brown bands). Verify PA2 is actually going HIGH by checking with the debugger.
 8. **Checksum always fails** → Your bit timing is probably off. The most critical measurement is distinguishing a "0" bit HIGH period (26μs) from a "1" bit HIGH period (70μs). If your microsecond delay isn't calibrated, you'll misread bits. Try measuring the HIGH duration by counting timer ticks instead of using a fixed delay.
-

Resources to Figure It Out

- **STM32F103C8T6 datasheet** — Google it, read the GPIO and timer sections
 - **DHT11 datasheet** — Google "DHT11 datasheet PDF", protocol timing diagram is on page 5
 - **STM32 HAL documentation** — Google "STM32 HAL driver user manual UM1850", or read the header files directly in your Drivers/STM32F1xx_HAL_Driver/Inc/ folder — they have comments explaining every function
 - **FreeRTOS source code** — You're already reading this, the bitwise patterns you see there apply directly
 - **STM32 CLI tools documentation** — Run the help commands to discover build, flash, and debug options
-

After This Works

Come back. Project 2 will add an OLED display from your kit and replace the UART output with on-screen display. Then Project 3 adds FreeRTOS with separate tasks for reading, displaying, and alerting.